[Flutter] Future, async, await, .then()

Flutter에서 `Future`는 **비동기 처리**를 위한 핵심 개념입니다. 쉽게 말해서, *조금 시간이 걸리는 일의 결과를 미래에 받을 수 있는 약속(promise)*이라고 생각하면 됩니다.

✅ Future란?

- `Future`는 **미래에 완료될 작업의 결과**를 나타냅니다.
- 예를 들어, **서버에서 데이터를 받아오기**, **파일 읽기**, **데이터베이스 조회** 같은 작업은 시간이 걸리기 때문에 바로 결과를 받을 수 없습니다.
- 이때 `Future`를 사용해서 "작업이 끝나면 알려줘!"라고 해놓는 거예요.

📦 비유로 이해하기

> 치킨 배달 주문 비유

> 1. 너가 치킨을 시켰어.
> 2. 바로 치킨이 오진 않지? 하지만 언젠간 도착할 거야.
> 3. `Future<치킨>` = 치킨이 *언젠간* 도착한다는 "약속"
> 4. 배달 완료되면 알려줘! = `await` 혹은 `.then()`을 통해 처리

📌 사용법 요약

1. `async` / `await` 방식 (가장 많이 씀)

```
Future<String> getName() async {
  await Future.delayed(Duration(seconds: 2));
  return "홍길동";
}

void main() async {
  print("이름을 가져오는 중...");
  String name = await getName(); // 결과 기다림
  print("이름: $name");
}
```

> await은 결과가 나올 때까지 기다려줌.
>
> `async`는 해당 함수가 비동기 함수라는 뜻.

---

2. `.then()` 방식

```
Future<String> getName() {
  return Future.delayed(Duration(seconds: 2), () => "홍길동");
}

void main() {
```

```
  print("이름을 가져오는 중...");
  getName().then((name) {
    print("이름: $name");
  });
}
```

> then()은 작업이 끝나고 나서 결과를 받아서 처리할 수 있음.

📍 Future가 왜 필요할까?

- 네트워크, DB 작업은 **시간이 걸림** → 그동안 앱이 **멈추면 안 됨**
- Flutter는 **싱글 스레드**이므로, 이런 작업은 **비동기 처리**를 꼭 해야 함.
- `Future`는 이런 작업을 *효율적이고 안전하게 처리할 수 있게 해줌.*

🧠 Future 관련 자주 나오는 것들

개념	설명
`Future.delayed()`	일정 시간 후 결과 리턴
`async`	비동기 함수 선언 키워드
`await`	Future의 결과 기다림
`then()`	Future 완료 후 실행할 콜백 등록
`catchError()`	에러 발생 시 처리
`FutureBuilder`	Flutter 위젯에서 Future를 다룰 때 사용

🎯 정리 한 줄 요약

> Future는 시간이 걸리는 작업의 결과를 미래에 받을 수 있게 해주는 약속이자, 비동기 프로그래밍의 핵심이다.